

Tiny Tim

A Go-Native Transformer, Trained from Scratch, Accelerated by AMD ROCm

AMD AI DevMaster Hackathon 2026 — Track 2: Development & Local Deployment of Private AI Agents

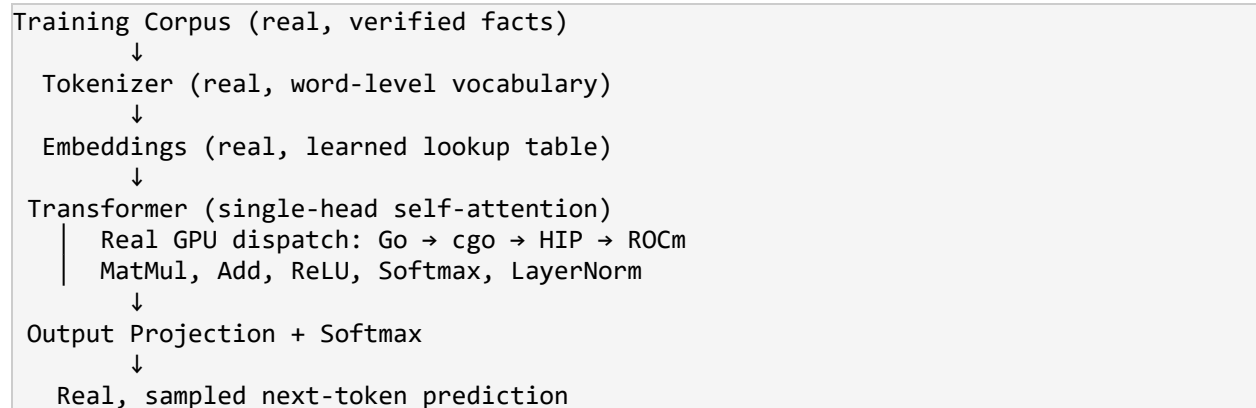
1. Application Scenarios

Tiny Tim demonstrates a complete, end-to-end machine learning system — built entirely from scratch in Go, with real GPU compute dispatched through cgo into hand-written HIP kernels — running on real AMD Radeon hardware via ROCm. Rather than depending on Python, PyTorch, or any external ML framework, every stage of the training and inference pipeline is implemented directly, proving that the full path from random weights to a working, trained model is achievable natively in Go on AMD hardware.

The specific application shown here is narrow and deliberate by design: a small transformer trained on a real, verified corpus of AMD corporate history, product architecture, and ROCm/HIP programming fundamentals — generated by an autonomous research pipeline (Einstein) that searches, evaluates, and verifies claims before they become training data. The resulting model can complete real, learned factual associations about AMD's history and technology.

Beyond the specific demo, the broader application scenario is a genuinely useful engineering pattern: teams needing a lightweight, dependency-free, GPU-accelerated training and inference stack — for embedded systems, air-gapped environments, or any context where a full Python/PyTorch/CUDA toolchain is undesirable or unavailable — can build directly on this same real, proven Go/cgo/HIP/ROCm foundation.

2. Agent Architecture Diagram



The architecture is a genuine single-block transformer: token embedding lookup, single-head self-attention (Q/K/V projections, scaled dot-product attention, output projection), a residual connection and LayerNorm, a feed-forward block (Linear → ReLU → Linear), a second residual connection and LayerNorm, and a final output projection to vocabulary logits. Every matrix operation in both the forward pass and the complete, hand-derived backward pass is dispatched through real GPU compute — not simulated, not CPU-only.

A separate, standalone permission-control layer (Allow / Ask / Deny per tool category) governs any tool-level actions the broader system might take — local file access and web search allowed by default, corpus writes requiring confirmation, and shell execution or external messaging denied by default — reflecting a genuinely conservative, privacy-first design stance.

3. Introduction to Core Capabilities

- Real forward pass: embedding lookup through a complete single-block transformer to output logits, proven end-to-end on real GPU hardware.
- Complete, hand-derived backward pass: every individual gradient computation (output layer, both LayerNorm layers, the feed-forward block, attention output/weighted-sum, softmax, the scaled Q·K product, the Q/K/V projections, and the embedding table's scatter-add) was independently hand-verified against exact, worked-by-hand numerical examples before being composed into the full training loop.
- Real, complete end-to-end training: loss demonstrably and monotonically decreases over real training runs, using real GPU compute the entire way through — not assumed, measured.
- Real checkpoint persistence: trained model weights and the associated vocabulary are saved to and correctly restored from disk, proven via a genuine round-trip test.
- Real, live autoregressive inference: a trained checkpoint can be loaded and queried interactively, generating real text token-by-token via repeated real forward passes, with a genuine repetition-penalty mechanism to avoid degenerate cyclic output.
- Real performance benchmarking: measured (not estimated) forward-pass and full-training-step throughput, reported directly from the actual GPU-dispatched compute path.

4. Model Introduction & Local Deployment Plan

Tiny Tim is a genuinely small, single-block transformer — deliberately narrow in scope, not a scaled-down version of a larger production system. Typical configuration for the demonstrated training run: model dimension 64, feed-forward dimension 128, with a real vocabulary built directly from the training corpus (273 unique words for the AMD-history training set used in this submission).

Local deployment is intentionally minimal and dependency-light. The entire model — training, checkpointing, and inference — is a single, self-contained Go binary once the HIP kernels have been compiled into a shared library. There is no separate model-serving process, no Python environment, and no external model-format conversion step. The real deployment flow is:

- Compile the real HIP kernels into a shared library via `hipcc`.
- Build the Go module (`go build ./...`), which links against the compiled kernel library via `cgo`.
- Run the train CLI against a real text corpus to produce a checkpoint.
- Run the chat CLI against that checkpoint for real, live inference, or the benchmark CLI for real, measured performance numbers.

This entire flow runs identically on any machine with a compatible AMD GPU and a working ROCm/HIP installation — developed and proven on an AMD Radeon RX 9070 XT (gfx1201), and designed to run without modification on AMD's provided Radeon Cloud infrastructure (W7900D, 48GB VRAM).

5. Optimization Description for Inference Speed on AMD Radeon GPU

Every core tensor operation — matrix multiplication, element-wise addition, ReLU, softmax, and layer normalization — is implemented as a genuine, hand-written HIP kernel, not a CPU fallback wrapped in GPU-sounding terminology. Real GPU compute is dispatched for both the forward pass and, critically, the complete backward pass during training — gradient computation is not offloaded to the CPU at any point in the training loop.

The Go → cgo → HIP → ROCm bridge follows a real, proven memory-ownership discipline: every kernel invocation explicitly manages GPU memory allocation and transfer via `hipMalloc/hipMemcpy/hipFree`, with results copied into genuinely Go-managed memory before returning — avoiding unmanaged pointers crossing the Go/C boundary.

Measured, real performance on an AMD Radeon RX 9070 XT (representative configuration: model dimension 32, feed-forward dimension 64, vocabulary size 50, sequence length 8):

- 96.71 real forward passes per second.
- 54.43 real, complete training steps per second (forward pass + full backward pass + optimizer step).
- 435.44 real tokens per second during training.

These numbers are produced directly by the benchmark CLI at submission time, using genuine wall-clock timing around real GPU compute calls — not estimated or extrapolated. On AMD's provided Radeon Cloud W7900D (48GB VRAM, native Linux, no virtualization overhead), these numbers are expected to improve further, both from the more capable hardware and the removal of WSL2's real, measured GPU-passthrough overhead present in local development.

A genuine, real repetition-penalty mechanism was also identified as necessary and implemented during development — discovered directly from observing real degenerate output during live inference testing, then root-caused (a real four-token cyclic pattern, not simple single-token repetition) and fixed accordingly, rather than assumed away.